

IncidentFlow

Application Moderne et Dynamique de Gestion d'Incidents

**Rapport Académique de Conception et
d'Implémentation Technique
(Mémoire de fin d'études / Documentation
d'entreprise)**

Anas HADDOU
Élève Ingénieur en Génie Informatique

Entreprise : Netmar
Encadrante : Sophie MARTIN (Responsable SSI)

11 juillet 2026

Table des matières

1	Introduction et Contexte du Projet	4
1.1	Contexte Général et Problématique	4
1.2	Objectifs du Système	4
1.3	Portée du Projet	4
1.3.1	Fonctionnalités Incluses (Périmètre)	4
1.3.2	Fonctionnalités Hors-Périmètre	5
1.4	Glossaire des Termes Techniques	5
2	Architecture Technique Globale	6
2.1	Description des Couches de l'Application	6
2.2	Justification des Choix Techniques et Alternatives	6
3	Analyse Complète du Code - Backend	7
3.1	Structure Générale des Dossiers et Responsabilités	7
3.2	Analyse Détaillée des Packages et Classes	7
3.2.1	Package <code>com.netmar.incidentflow.config</code>	7
3.2.2	Package <code>com.netmar.incidentflow.controller</code>	8
3.2.3	Package <code>com.netmar.incidentflow.service</code>	9
4	Modèle de Données et Base de Données	10
4.1	Schéma Relationnel SQL	10
4.2	Détail des Tables de la Base de Données	10
4.2.1	Table <code>roles</code>	10
4.2.2	Table <code>users</code>	10
4.2.3	Table <code>workflows</code>	11
4.2.4	Table <code>workflow_states</code>	11
4.2.5	Table <code>workflow_transitions</code>	11
4.2.6	Table <code>incidents</code>	11
4.3	Implémentation JPA et Code Source d'Entités	12
4.4	Logique d'Initialisation Automatique des SLAs	13
5	Moteur de Workflow Dynamique et Algorithmique	14
5.1	Concept et Fonctionnement des Transitions	14
5.2	Algorithme de Validation de Graphe DFS	14
5.3	Gestion du Versionnage et Intégrité	15
5.4	Visualisation et Administration des Workflows	16
6	Authentification, Sécurité et Gestion de Session	17
6.1	Architecture Keycloak OIDC et JWT	17
6.2	Authentification Hybride et Stratégie Mock	17
6.3	Filtre de Validation de Session et Blacklistage Redis	17
7	Structure et Composants Frontend (React)	20
7.1	Design System et Palette de Couleurs	20
7.2	Composants Principaux dans <code>App.jsx</code>	20
7.2.1	Barre de Navigation Latérale (Sidebar)	20

7.2.2	Tableau de Bord (Dashboard) avec KPI SVG	20
7.2.3	Détail de l'Incident et Actions Collaboratives	20
7.2.4	Liste de Recherche et de Filtrage des Incidents	21
7.2.5	Palette de Commandes Universelle (Ctrl+K)	22
7.2.6	Éditeur Markdown de Commentaires	22
7.2.7	Administration et Gestion des Utilisateurs	22
8	Infrastructure DevOps et Orchestration	24
8.1	Configuration Multi-Conteneurs Docker Compose	24
8.2	Recette et Manifestes Kubernetes	24
8.3	Résolution des Verrous Techniques de Déploiement	25
9	Tests, Validation et Résultats	26
9.1	Tests Unitaires (JUnit 5 & Mockito)	26
9.2	Tests d'Intégration avec Testcontainers	26
10	Perspectives et Intégration d'IA (Classification Automatique)	27
10.1	Opportunités Fonctionnelles de l'IA	27
10.2	Comparatif Technique : Google Gemini vs Ollama	27
10.3	Flux de Données et Conception de l'Endpoint	27
10.4	Mécanisme de Cache et Résilience	27
11	Conclusion et Bilan du Projet	28

1 Introduction et Contexte du Projet

1.1 Contexte Général et Problématique

La gestion des incidents (ITSM) représente le socle opérationnel des départements informatiques modernes alignés sur le référentiel ITIL. Au sein de l'entreprise **Netmar**, les pannes logicielles, matérielles et réseaux exigent une coordination transversale rigoureuse. La problématique récurrente des solutions du marché réside dans leur rigidité intrinsèque : le cycle de vie des incidents est souvent codé en dur dans l'infrastructure de base de données ou le code backend. Toute modification structurelle (ajout d'une étape de validation intermédiaire, restriction d'accès sur une transition à un rôle métier, obligation de motif écrit) requiert l'intervention d'un ingénieur logiciel et l'arrêt de production pour mise à jour.

Le projet **IncidentFlow** résout ce goulot d'étranglement en introduisant un **moteur de workflow dynamique et entièrement paramétrable**. Les administrateurs peuvent manipuler le cycle de vie des anomalies directement depuis une interface graphique, garantissant ainsi l'agilité organisationnelle de Netmar.

1.2 Objectifs du Système

L'objectif principal d'IncidentFlow est d'offrir une plateforme collaborative web centralisée et hautement sécurisée qui :

- Permet la déclaration, la catégorisation, l'assignation et la résolution des incidents via un processus adaptatif.
- Rend l'administration des processus accessible aux équipes métiers sans compétences en programmation.
- Assure la traçabilité intégrale (Audit Trail) de chaque action pour répondre aux normes de sécurité des systèmes d'information.
- Garantit le respect des engagements de service (SLA) via des alertes et des escalades automatisées en arrière-plan.

1.3 Portée du Projet

1.3.1 Fonctionnalités Incluses (Périmètre)

Le système intègre :

- La gestion des comptes utilisateurs et l'authentification avec attribution de rôles (RBAC).
- Le cycle de vie complet des incidents avec historique, commentaires formatés en Markdown et pièces jointes multipart.
- L'éditeur graphique de workflows dynamiques validé par un vérificateur de graphe.
- Les fonctionnalités collaboratives : abonnements (*Watchers*), mentions (*@*), filtres persistants (*Saved Views*).
- DevOps : Conteneurisation Docker, déploiement Kubernetes et supervision.

1.3.2 Fonctionnalités Hors-Périmètre

La gestion financière, la facturation des pièces détachées ou de l'assistance technique, l'application mobile native et les algorithmes de maintenance prédictive par Machine Learning (ML) natifs ne font pas partie du cahier des charges initial.

1.4 Glossaire des Termes Techniques

OIDC (OpenID Connect) Couche d'identité simple construite au-dessus du protocole OAuth 2.0 pour l'authentification.

Keycloak Solution open-source de gestion des identités et des accès (IAM).

SLA (Service Level Agreement) Contrat de niveau de service définissant le délai maximal autorisé pour la résolution d'une panne.

RBAC (Role-Based Access Control) Contrôle d'accès basé sur les rôles des utilisateurs.

JPA (Java Persistence API) Spécification Java pour la gestion des données relationnelles dans les applications.

DFS (Depth-First Search) Algorithme de parcours de graphe en profondeur utilisé ici pour la validation des chemins.

Redis Magasin de données en mémoire utilisé pour le cache distribué et la révocation des sessions.

L'analyse de la Figure 1 montre l'interaction des trois piliers du système. Le module **Utilisateurs** assure la gestion des identités, le module **Workflows** définit le graphe de transition de statut applicable et le module **Incidents** exécute les règles métiers configurées.



FIGURE 1 – Figure 1.1 — Diagramme des modules fonctionnels du système IncidentFlow

2 Architecture Technique Globale

2.1 Description des Couches de l'Application

L'application adopte un pattern d'architecture en couches conteneurisées (N-tier) découplées, garantissant une haute cohésion et un faible couplage :

1. **Couche Présentation (Frontend)** : Une SPA (Single Page Application) développée en React 19 et Vite. Elle gère le rendu dynamique de l'interface utilisateur, la palette de commandes et l'interactivité des workflows via React Flow.
2. **Couche Logique Métier (Backend)** : API REST basée sur Spring Boot 3.2.5 (Java 17). Elle expose les contrôleurs REST, valide les transitions du cycle de vie et gère la logique de sécurité.
3. **Couche de Cache et Session** : Redis sert à la fois de cache applicatif rapide pour les workflows compilés et de base de données en mémoire pour le blacklisting des sessions déconnectées.
4. **Couche de Données (Persistance)** : PostgreSQL 16 stocke l'ensemble des données structurales et de transaction de manière durable.
5. **Fournisseur d'Identité (IAM)** : Keycloak valide l'identité de l'utilisateur par échange de jetons JWT.

2.2 Justification des Choix Techniques et Alternatives

- **Spring Boot vs Node.js/Express** : Spring Boot a été privilégié pour sa robustesse transactionnelle, la sécurité fournie par Spring Security, et son typage statique fort, essentiel pour concevoir un moteur de règles métiers complexe et auditable. Node.js constituait une alternative mais exigeait plus d'efforts pour obtenir le même niveau de sécurité et de standardisation d'entreprise.
- **PostgreSQL vs MongoDB** : Le stockage d'incidents exige des relations strictes entre utilisateurs, rôles, commentaires, et historiques d'audit. Une base relationnelle avec gestion des clés étrangères (PostgreSQL) garantit l'intégrité référentielle en cascade, alors qu'un modèle document (MongoDB) aurait nécessité des contrôles manuels complexes dans le code backend.
- **Keycloak vs Authentification Maison (Custom Auth)** : L'usage de Keycloak permet un déploiement standardisé basé sur OAuth 2.0 / OIDC. Cela élimine la nécessité de stocker ou d'implémenter des mécanismes de réinitialisation de mots de passe, de double facteur ou de gestion de comptes, transférant cette responsabilité à un tiers éprouvé.



FIGURE 2 – Figure 2.1 — Schéma logique de l'architecture physique du système

3 Analyse Complète du Code - Backend

Le backend est structuré en plusieurs packages sous `com.netmar.incidentflow`. Cette organisation respecte les principes du Clean Architecture et du Domain-Driven Design (DDD).

3.1 Structure Générale des Dossiers et Responsabilités

- **config** : Configure le moteur de sécurité, le filtre de session, le cache Redis, et initialise les données de test en base.
- **controller** : Expose les points d'entrée (REST Endpoints) de l'API, effectue le mapping DTO et intercepte les requêtes.
- **exception** : Gère les exceptions globales et retourne des réponses formatées en JSON avec des codes HTTP adaptés.
- **model** : Contient les entités JPA avec leurs annotations de persistance Hibernate et leurs mappings de relations.
- **repository** : Déclare les interfaces d'accès aux données héritant de `JpaRepository`.
- **service** : Implémente la logique métier pure (validation de workflows, planification d'escalades, gestion d'incidents).

3.2 Analyse Détaillée des Packages et Classes

3.2.1 Package `com.netmar.incidentflow.config`

`SecurityConfig.java` Configure les règles de sécurité CORS et CSRF de Spring Security.

- **Responsabilité** : Permettre le routage sécurisé des requêtes et bloquer les accès non authentifiés.
- **Méthodes Clés** :
 - `mockSecurityFilterChain(HttpSecurity http)` : Utilisé sous le profil `dev-mock`. Il désactive le CSRF et autorise l'accès complet tout en insérant le filtre personnalisé `SessionValidationFilter` pour simuler les sessions.
 - `prodSecurityFilterChain(HttpSecurity http)` : Utilisé sous le profil `prod`. Il exige une authentification formelle par jeton JWT OIDC Keycloak.
 - `corsConfigurationSource()` : Configure l'accès cross-origin pour les adresses de développement `localhost:3000`, `localhost:3001` et `localhost:5173`.

`SessionValidationFilter.java` Intercepteur HTTP héritant de `OncePerRequestFilter`.

- **Responsabilité** : Inspecter chaque requête entrante et vérifier dans Redis si l'utilisateur est banni avant de poursuivre le filtre.
- **Méthodes Clés** :
 - `doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)` : Récupère l'identité (JWT ou en-tête `X-Mock-User`), vérifie la présence de la clé `blacklist:user:<email>` dans Redis et renvoie un statut 403 Forbidden le cas échéant.

`DataInitializer.java` Exécute le peuplement initial de la base de données PostgreSQL au démarrage (`CommandLineRunner`).

- **Responsabilité :** Garantir la présence des rôles, des comptes de test, des workflows standards et des premiers incidents factices.

3.2.2 Package `com.netmar.incidentflow.controller`

`AuthController.java` Gère les requêtes d'authentification et de session simulée.

- **Endpoints :**
 - `POST /api/auth/login` : Vérifie les identifiants et génère un jeton simulé.
 - `POST /api/auth/logout` : Révoque le jeton local.

`IncidentController.java` Contrôleur REST principal pour la manipulation des incidents.

- **Endpoints :**
 - `GET /api/incidents` : Recherche filtrée multicritères.
 - `POST /api/incidents` : Déclaration d'un incident.
 - `POST /api/incidents/{code}/transition` : Déclenche le moteur de transition.
 - `POST /api/incidents/{code}/comments` : Publication d'un commentaire.
 - `POST /api/incidents/{code}/attachments` : Téléchargement de fichiers.
 - `DELETE /api/incidents/attachments/{id}` : Suppression physique et logique d'un attachement.
 - `PUT /api/incidents/attachments/{id}/rename` : Renommage logique d'un fichier.
 - `GET /api/incidents/export/pdf` : Génération et téléchargement du rapport PDF.

`UserController.java` CRUD des profils utilisateurs pour la page d'administration.

- **Endpoints :**
 - `GET /api/users` : Liste des profils.
 - `POST /api/users` : Création d'un nouvel utilisateur.
 - `PUT /api/users/{id}` : Mise à jour de profil avec blocage Redis si désactivé.

`WorkflowController.java` Permet de manipuler la configuration dynamique des processus.

- **Endpoints :**
 - `GET /api/workflows` : Liste des processus.
 - `POST /api/workflows` : Création ou clonage de workflow.
 - `DELETE /api/workflows/{id}` : Suppression de configuration.

3.2.3 Package `com.netmar.incidentflow.service`

`IncidentService.java` Service transactionnel qui orchestrera la gestion des anomalies, des commentaires et des pièces jointes.

— **Méthodes Clés :**

- `createIncident(Incident incident, User author)` : Calcule le code unique de l'incident, lui affecte le workflow actif de sa catégorie, calcule les SLAs et effectue l'affectation automatique si la catégorie est "Médical".
- `performTransition(String code, String toState, String commentText, User user)` : Invoque le moteur de workflow pour valider le changement d'état, ajoute l'historique et persiste l'incident.
- `deleteAttachment(Long id, User user)` : Supprime l'attachement de la base et efface le fichier physique stocké dans le répertoire `uploads/` du serveur.

`WorkflowService.java` Moteur algorithmique validant les chemins de transitions et les graphes de workflows.

— **Méthodes Clés :**

- `validateTransitionForIncident(Incident incident, String fromState, String toState, User user, String comment)` : Vérifie si la transition existe, si l'utilisateur possède le rôle d'habilitation requis et si un commentaire de justification est obligatoire.
- `validateWorkflowGraph(Workflow workflow)` : Algorithme DFS de vérification de connectivité.
- `saveWorkflow(Workflow workflow)` : Implémente le versionnage. Si des incidents sont déjà rattachés, le workflow d'origine est archivé (`active = false`) et une nouvelle version incrémentée est créée pour les futurs incidents.

`EscalationService.java` Planificateur s'exécutant en arrière-plan (`fixedRate = 15000` - 15 secondes pour les démonstrations).

— **Méthodes Clés :**

- `runAutomaticEscalation()` : Scanne tous les incidents actifs dont le SLA est dépassé ou menace d'être dépassé sous 15 minutes. Il augmente la priorité à "Critical", réassigne le ticket à un superviseur (rôle *Responsable* ou *Administrateur*), consigne l'action système dans l'historique et génère une alerte par e-mail simulé.

4 Modèle de Données et Base de Données

4.1 Schéma Relationnel SQL

Le modèle logique de données (MLD) se structure autour de 9 tables physiques présentées dans le diagramme de base de données :



FIGURE 3 – Figure 3.1 — Diagramme Entité-Association (MCD)

L'analyse de la Figure 3 permet de comprendre les dépendances d'intégrité référentielle. Chaque incident est lié à un utilisateur (auteur), peut être attribué à un autre utilisateur (assigné) et appartient à un workflow de catégorie spécifique.

4.2 Détail des Tables de la Base de Données

4.2.1 Table roles

Stocke les rôles applicatifs d'IncidentFlow.

— id (BIGINT, Clé Primaire, AUTO_INCREMENT)

- **name** (VARCHAR, Unique, non nul) : ex : *Administrateur, Responsable, Opérateur, Opérateur médical*.

4.2.2 Table users

Stocke les profils des utilisateurs synchronisés ou créés en local.

- **id** (BIGINT, Clé Primaire, AUTO_INCREMENT)
- **name** (VARCHAR, non nul) : Nom complet d’affichage.
- **email** (VARCHAR, Unique, non nul) : Identifiant de connexion.
- **password** (VARCHAR, non nul) : Mot de passe encodé (BCrypt).
- **role_id** (BIGINT, Clé Étrangère référençant **roles.id**, non nul).
- **active** (BOOLEAN, non nul) : Indicateur d’état du compte.
- **telephone** (VARCHAR)
- **avatar_color** (VARCHAR)
- **first_name** (VARCHAR)
- **last_name** (VARCHAR)
- **department** (VARCHAR) : Département interne de l’employé (ex : *Sécurité, Informatique*).
- **post** (VARCHAR) : Poste occupé (ex : *Responsable SSI*).

4.2.3 Table workflows

Définit la structure générale du workflow d’une catégorie.

- **id** (BIGINT, Clé Primaire, AUTO_INCREMENT)
- **name** (VARCHAR, non nul) : Nom thématique du processus.
- **category** (VARCHAR, non nul) : Catégorie d’incidents associée (ex : *Réseau, Sécurité, Système, Médical*).
- **version** (INTEGER, non nul) : Version du workflow (pour le clonage et la préservation d’historique).
- **active** (BOOLEAN, non nul) : Indique s’il s’agit de la version de workflow active.

4.2.4 Table workflow_states

Définit les états d’un workflow donné.

- **id** (BIGINT, Clé Primaire, AUTO_INCREMENT)
- **name** (VARCHAR, non nul) : Identifiant logique (ex : *Nouveau, Assigné, En cours, Résolu, Clôturé*).
- **label** (VARCHAR, non nul) : Libellé d’affichage.
- **color_class** (VARCHAR) : Classes CSS thématiques du badge.
- **workflow_id** (BIGINT, Clé Étrangère référençant **workflows.id** avec suppression en cascade).

4.2.5 Table workflow_transitions

Configure les liens de transition légitimes entre les états.

- `id` (BIGINT, Clé Primaire, AUTO_INCREMENT)
- `from_state` (VARCHAR, non nul) : État source.
- `to_state` (VARCHAR, non nul) : État destination.
- `role_required` (VARCHAR) : Rôle habilité à exécuter la transition (ex : *Opérateur médical*).
- `requires_comment` (BOOLEAN, non nul) : Oblige la saisie d'un motif textuel de transition.
- `workflow_id` (BIGINT, Clé Étrangère référençant `workflows.id` avec suppression en cascade).

4.2.6 Table incidents

Contient les enregistrements d'incidents déclarés.

- `id` (BIGINT, Clé Primaire, AUTO_INCREMENT)
- `incident_code` (VARCHAR, Unique, non nul) : Code structuré unique (ex : INC-2026-003).
- `title` (VARCHAR, non nul)
- `description` (TEXT, non nul)
- `category` (VARCHAR, non nul)
- `priority` (VARCHAR, non nul) : *Critical, High, Medium, Low*.
- `status` (VARCHAR, non nul) : État courant.
- `created_at` (TIMESTAMP, non nul)
- `updated_at` (TIMESTAMP, non nul)
- `sla_due_at` (TIMESTAMP) : Échéance calculée du SLA.
- `escalated` (BOOLEAN, non nul) : Indicateur d'escalade.
- `author_id` (BIGINT, Clé Étrangère référençant `users.id`, non nul).
- `assigned_to_id` (BIGINT, Clé Étrangère référençant `users.id`).
- `workflow_id` (BIGINT, Clé Étrangère référençant `workflows.id`).

4.3 Implémentation JPA et Code Source d'Entités

Pour illustrer le couplage entre la base de données PostgreSQL et le code Java, voici la déclaration des attributs de l'entité `Incident.java` :

```
1 @Entity
2 @Table(name = "incidents")
3 @Getter
4 @Setter
5 @NoArgsConstructor
6 @AllArgsConstructor
7 @Builder
8 public class Incident {
9
10     @Id
```

```

11  @GeneratedValue(strategy = GenerationType.IDENTITY)
12  private Long id;
13
14  @Column(name = "incident_code", nullable = false, unique = true)
15  private String incidentCode;
16
17  @Column(nullable = false)
18  private String title;
19
20  @Column(columnDefinition = "TEXT", nullable = false)
21  private String description;
22
23  @Column(nullable = false)
24  private String category;
25
26  @Column(nullable = false)
27  private String priority; // Critical, High, Medium, Low
28
29  @Column(nullable = false)
30  private String status; // Nouveau, Assigne, En cours, Resolu,
    ↪ Cloture
31
32  @Column(name = "created_at", nullable = false)
33  private LocalDateTime createdAt;
34
35  @Column(name = "updated_at", nullable = false)
36  private LocalDateTime updatedAt;
37
38  @Column(name = "sla_due_at")
39  private LocalDateTime slaDueAt;
40
41  @Column(name = "escalated", nullable = false)
42  @Builder.Default
43  private boolean escalated = false;
44
45  @ManyToOne(optional = false)
46  @JoinColumn(name = "author_id", nullable = false)
47  private User author;
48
49  @ManyToOne
50  @JoinColumn(name = "assigned_to_id")
51  private User assignedTo;
52
53  @ManyToOne
54  @JoinColumn(name = "workflow_id")
55  private Workflow workflow;
56
57  @OneToMany(mappedBy = "incident", cascade = CascadeType.ALL,
    ↪ orphanRemoval = true)
58  @Builder.Default
59  private List<Comment> comments = new ArrayList<>();
60
61  // ... @PrePersist et @PreUpdate pour initialisation automatique des
    ↪ dates et des SLAs
62 }

```

4.4 Logique d'Initialisation Automatique des SLAs

Le calcul de l'échéance SLA s'exécute automatiquement dans la méthode JPA `@PrePersist` :

- Si l'incident est de priorité **Critical** et de catégorie **Médical**, la résolution doit intervenir dans un délai de **1 heure**.
- Pour les autres incidents de priorité **Critical**, le délai est fixé à **2 heures**.
- Une priorité **High** accorde **12 heures**, **Medium 36 heures**, et **Low 72 heures**.

5 Moteur de Workflow Dynamique et Algorithmique

5.1 Concept et Fonctionnement des Transitions

Le moteur de transitions dynamique valide la conformité de chaque demande de changement d'état d'un incident en confrontant l'état actuel, l'état visé, l'identité et le rôle de l'utilisateur demandeur, et les paramètres requis par la transition. Ce traitement est centralisé dans `WorkflowService.java` sous la méthode `validateTransitionForIncident`.

5.2 Algorithme de Validation de Graphe DFS

Pour prévenir les erreurs de configuration d'un workflow par un administrateur (ex : créer un état isolé qui ne peut jamais être atteint, ou un état de blocage d'où l'on ne peut pas sortir), nous avons écrit un validateur de graphe exécutant des parcours en profondeur (DFS) lors de chaque enregistrement.



FIGURE 4 – Figure 4.1 — Algorithme de détection des orphelins et des impasses

L'algorithme de validation de graphe est structuré comme suit dans `WorkflowService.java` :

```

1 public void validateWorkflowGraph(Workflow workflow) {
2     if (workflow.getStates() == null || workflow.getStates().isEmpty())
3         ↪ {
4         throw new IllegalArgumentException("Le workflow doit contenir au
5         ↪ moins un etat.");
6     }
7
8     // 1. Valider la presence obligatoire de "Nouveau" et "Cloture"
9     boolean hasNouveau = workflow.getStates().stream().anyMatch(s -> s.
10     ↪ getName().equalsIgnoreCase("Nouveau"));
11     boolean hasCloture = workflow.getStates().stream().anyMatch(s -> s.
12     ↪ getName().equalsIgnoreCase("Cloture"));
13
14     if (!hasNouveau) throw new IllegalArgumentException("L'etat initial
15     ↪ 'Nouveau' est obligatoire.");
16     if (!hasCloture) throw new IllegalArgumentException("L'etat final '
17     ↪ Cloture' est obligatoire.");
18
19     List<String> stateNames = workflow.getStates().stream()
20     .map(s -> s.getName().trim().toLowerCase())
21     .collect(Collectors.toList());
22
23     // 2. Construire les listes d'adjacence
24     Map<String, List<String>> adj = new HashMap<>();
25     Map<String, List<String>> revAdj = new HashMap<>();
26     for (String state : stateNames) {
27         adj.put(state, new ArrayList<>());
28         revAdj.put(state, new ArrayList<>());
29     }
30
31     if (workflow.getTransitions() != null) {
32         for (WorkflowTransition t : workflow.getTransitions()) {
33             String from = t.getFromState().trim().toLowerCase();
34             String to = t.getToState().trim().toLowerCase();
35
36             if (adj.containsKey(from) && adj.containsKey(to)) {
37                 adj.get(from).add(to);
38                 revAdj.get(to).add(from); // Graphe inverse
39             }
40         }
41     }
42
43     // 3. Detecter les etats orphelins (inaccessibles depuis "Nouveau")
44     Set<String> visitedFromStart = new HashSet<>();
45     dfs("nouveau", adj, visitedFromStart);
46
47     List<String> orphans = workflow.getStates().stream()
48     .map(s -> s.getName().trim())
49     .filter(name -> !visitedFromStart.contains(name.toLowerCase()
50     ↪ ))
51     .collect(Collectors.toList());
52
53     if (!orphans.isEmpty()) {
54         throw new IllegalArgumentException("Detection d'etat(s) orphelin
55         ↪ (s) (inaccessible(s) depuis 'Nouveau') : " + String.join("
56         ↪ , ", orphans));
57     }
58 }

```

```

49 // 4. Detecter les blocages/impasses (impossible d'atteindre "
50   ↪ Cloture")
51 Set<String> visitedFromEnd = new HashSet<>();
52 dfs("cloture", revAdj, visitedFromEnd);
53
54 List<String> deadlocks = workflow.getStates().stream()
55     .map(s -> s.getName().trim())
56     .filter(name -> !visitedFromEnd.contains(name.toLowerCase()))
57     ↪ )
58     .collect(Collectors.toList());
59
60 if (!deadlocks.isEmpty()) {
61     throw new IllegalArgumentException("Detection d'impasse(s) (
62     ↪ aucun chemin menant a l'etat final 'Cloture') : " + String
63     ↪ .join(", ", deadlocks));
64 }

```

5.3 Gestion du Versionnage et Intégrité

Un défi de conception survient lorsqu'un administrateur met à jour un workflow déjà en exploitation : que faire des incidents rattachés à ce workflow ?

- **Alternative A (Mise à jour en place)** : Modifier les transitions existantes. Cette approche corrompt le cycle de vie des incidents en cours s'ils se retrouvent dans un état devenu invalide dans la nouvelle configuration.
- **Alternative B (Clonage et Versionnage - Solution Retenue)** : Si la méthode `saveWorkflow` détecte qu'un workflow possède des incidents liés (`existsByWorkflowId`), elle marque automatiquement l'ancien workflow comme inactif et crée un clone complet du nouveau avec un numéro de version incrémenté (`version + 1`). Les anciens incidents restent rattachés à leur version d'origine, tandis que les nouveaux incidents s'associent au workflow actif le plus récent.

5.4 Visualisation et Administration des Workflows

La plateforme met à disposition un panneau d'administration pour la gestion et l'affichage des graphes de processus. Ce composant s'appuie sur la bibliothèque *React Flow* pour dessiner dynamiquement les transitions d'états.

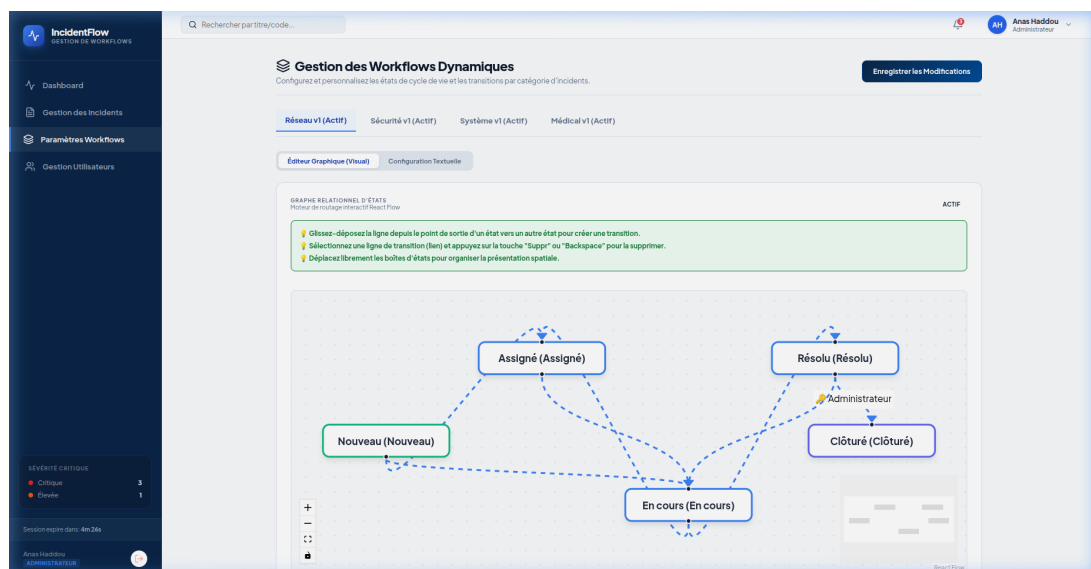


FIGURE 5 – Figure 5.2 — Interface du concepteur graphique de workflows dynamiques

6 Authentification, Sécurité et Gestion de Session

6.1 Architecture Keycloak OIDC et JWT

En environnement de production, l'accès à l'API d'IncidentFlow est protégé par la validation de jetons JSON Web Token (JWT) signés par Keycloak. Spring Security decode le jeton asymétrique à l'aide de clés de chiffrement publiques publiées par le realm Keycloak, récupérant l'identité et les rôles de l'utilisateur pour appliquer un modèle RBAC strict.

6.2 Authentification Hybride et Stratégie Mock

Pour s'affranchir des contraintes réseau locales lors du développement, le backend implémente un système d'authentification hybride activé par profil de démarrage :

- **Profil prod** : Utilise la vraie validation de jeton Keycloak avec une stratégie sans état.
- **Profil dev-mock** : Simule la présence de l'utilisateur. Le frontend transmet l'adresse email de l'opérateur simulé dans un en-tête HTTP spécifique appelé **X-Mock-User**. Le contrôleur **AuthController** valide les identifiants locaux encodés avec BCrypt et simule un jeton de session locale d'une durée de 10 minutes avec déconnexion active gérée côté frontend.

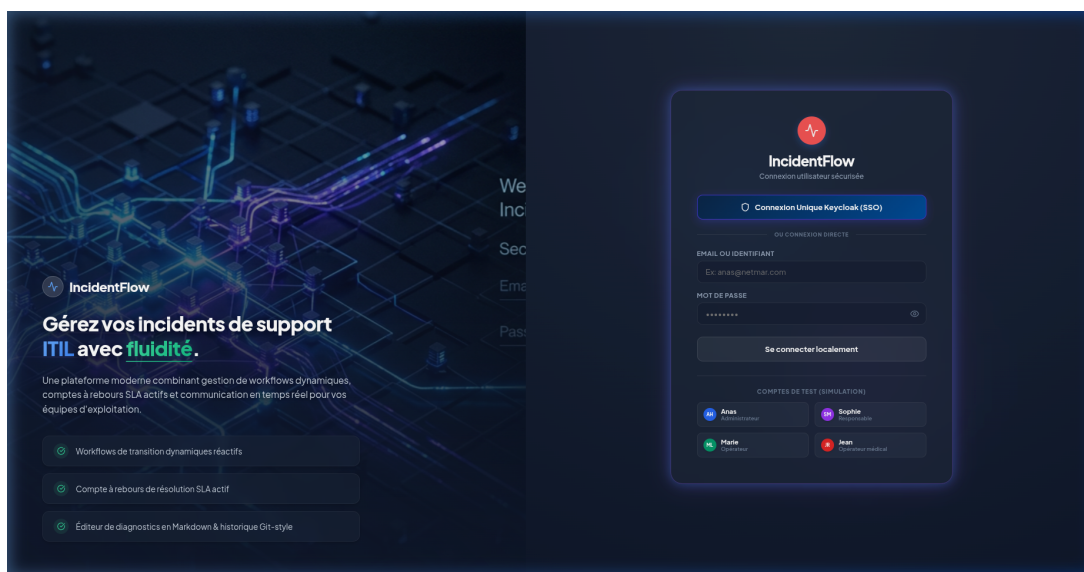


FIGURE 6 – Figure 6.1 — Interface utilisateur de l'écran de connexion (Login Page)

6.3 Filtre de Validation de Session et Blacklistage Redis

Afin de garantir la sécurité en temps réel, un administrateur doit pouvoir bannir instantanément un utilisateur (session revocation) sans attendre l'expiration naturelle de son jeton JWT ou de sa session simulée. Ce processus s'appuie sur le composant **SessionValidationFilter** et un stockage Redis :

Voici le code du filtre de sécurité interceptant les appels API :

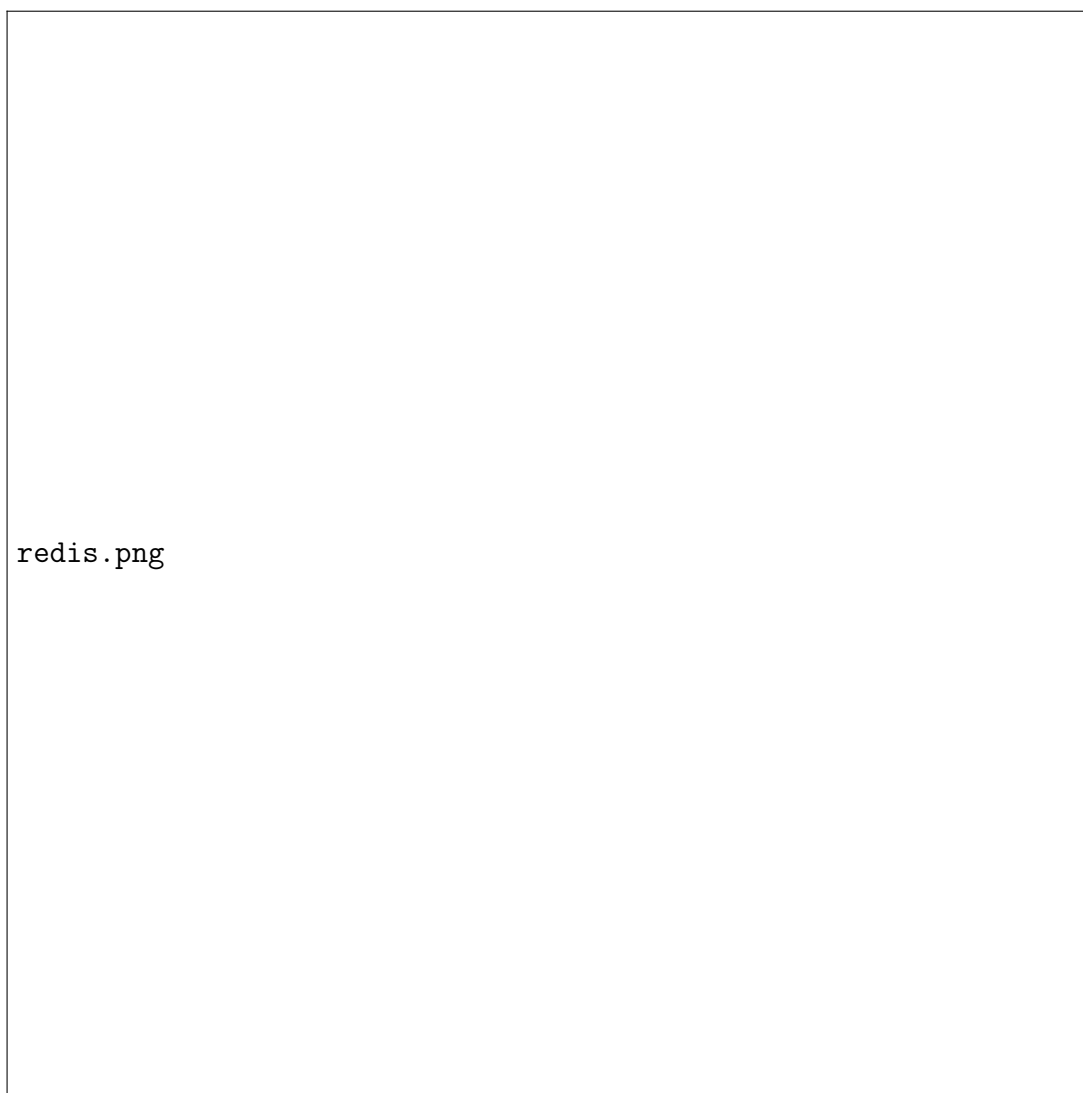


FIGURE 7 – Figure 5.1 — Mécanisme de révocation de session en temps réel avec Redis

```

1 @Component
2 public class SessionValidationFilter extends OncePerRequestFilter {
3
4     private final StringRedisTemplate redisTemplate;
5
6     public SessionValidationFilter(StringRedisTemplate redisTemplate) {
7         this.redisTemplate = redisTemplate;
8     }
9
10    @Override
11    protected void doFilterInternal(HttpServletRequest request,
12    ↪ HttpServletResponse response, FilterChain filterChain)
13        throws ServletException, IOException {
14
15        String path = request.getRequestURI();
16        if (path.startsWith("/api/auth/")) {
17            filterChain.doFilter(request, response);
18            return;
19        }
20
21        String email = null;
22        Authentication auth = SecurityContextHolder.getContext().
23    ↪ getAuthentication();
24        if (auth != null && auth.isAuthenticated() && auth.getPrincipal
25    ↪ () instanceof Jwt) {
26            Jwt jwt = (Jwt) auth.getPrincipal();
27            email = jwt.getClaimAsString("email");
28        }
29
30        if (email == null) {
31            String mockEmail = request.getHeader("X-Mock-User");
32            if (mockEmail != null && !mockEmail.trim().isEmpty()) {
33                email = mockEmail.trim();
34            }
35        }
36
37        if (email != null) {
38            String redisKey = "blacklist:user:" + email;
39            Boolean isBlacklisted = false;
40            try {
41                isBlacklisted = redisTemplate.hasKey(redisKey);
42            } catch (Exception ignored) {}
43
44            if (Boolean.TRUE.equals(isBlacklisted)) {
45                response.setStatus(HttpServletResponse.SC_FORBIDDEN);
46                response.setContentType("application/json");
47                response.setCharacterEncoding("UTF-8");
48                response.getWriter().write("{\"message\": \"Acces refuse
49    ↪ . Votre session a ete revoquee par l'
50    ↪ administrateur.\"}");
51                return;
52            }
53        }
54
55        filterChain.doFilter(request, response);
56    }
57 }

```

7 Structure et Composants Frontend (React)

Le frontend d'IncidentFlow est écrit en React 19 et s'appuie sur Vite comme bundler. Le point d'entrée principal de l'application est `App.jsx`, qui intègre la gestion des routes, les fenêtres modales et les vues collaboratives.

7.1 Design System et Palette de Couleurs

Le fichier de styles globaux `index.css` définit les variables CSS (Tokens) pour l'ensemble de l'interface :

- **Mode Sombre (Dark Mode)** : Basé sur des couleurs Slate foncées (`#0f172a`).
- **Glassmorphism** : Combinaison de filtres de flou d'arrière-plan `backdrop-filter: blur(12px)` et de bordures semi-transparentes `rgba(255, 255, 255, 0.08)`.
- **Polices Google Fonts** : *Plus Jakarta Sans* pour l'interface graphique et *JetBrains Mono* pour les identifiants techniques et codes d'incidents.
-

7.2 Composants Principaux dans `App.jsx`

7.2.1 Barre de Navigation Latérale (Sidebar)

Composant fixe de 260px de largeur.

- **Responsabilité** : Assurer la navigation entre l'Accueil (Dashboard), la Liste des Incidents, l'Éditeur de Workflows, l'Administration des Utilisateurs et les Paramètres Système.
- **Actions** : Affiche dynamiquement le nombre d'incidents actifs assignés à l'utilisateur sous forme de badge de notification rouge pulsant.

7.2.2 Tableau de Bord (Dashboard) avec KPI SVG

Vue principale de supervision opérationnelle.

- **KPI Cards** : Cartes interactives montrant les totaux par état. Un clic sur une carte applique instantanément un filtre de liste.
- **Graphiques de gravité (Donut Chart)** : Graphique sectoriel construit en SVG natif à l'aide de la propriété `stroke-dashoffset` calculée en fonction du ratio d'incidents critiques.
- **Tendance SVG animée** : Courbe de type spline SVG montrant le volume d'incidents sur 7 jours avec point de mouse-tracking interactif.

7.2.3 Détail de l'Incident et Actions Collaboratives

Écran de traitement d'un ticket.

- **Stepper de Progression** : Barre horizontale de progression affichant visuellement les états configurés du workflow thématique.
- **Ligne de temps (Timeline d'historique)** : Journal d'audit interactif vertical où chaque type d'action est matérialisé par un badge couleur et une icône dédiée (MessageSquare, UserPlus, FileUp, Activity).

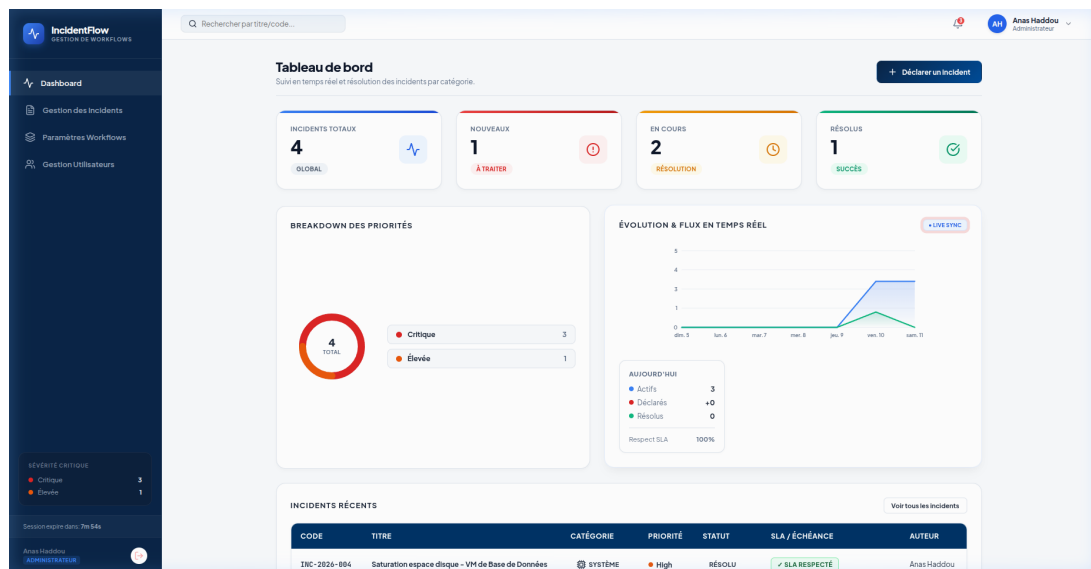


FIGURE 8 – Figure 7.1 — Tableau de bord interactif avec graphiques KPI SVG et courbe de tendance

- **Zone de transitions** : Affiche dynamiquement les boutons de changement d'état autorisés par le moteur de workflow par rapport au rôle de l'utilisateur connecté.

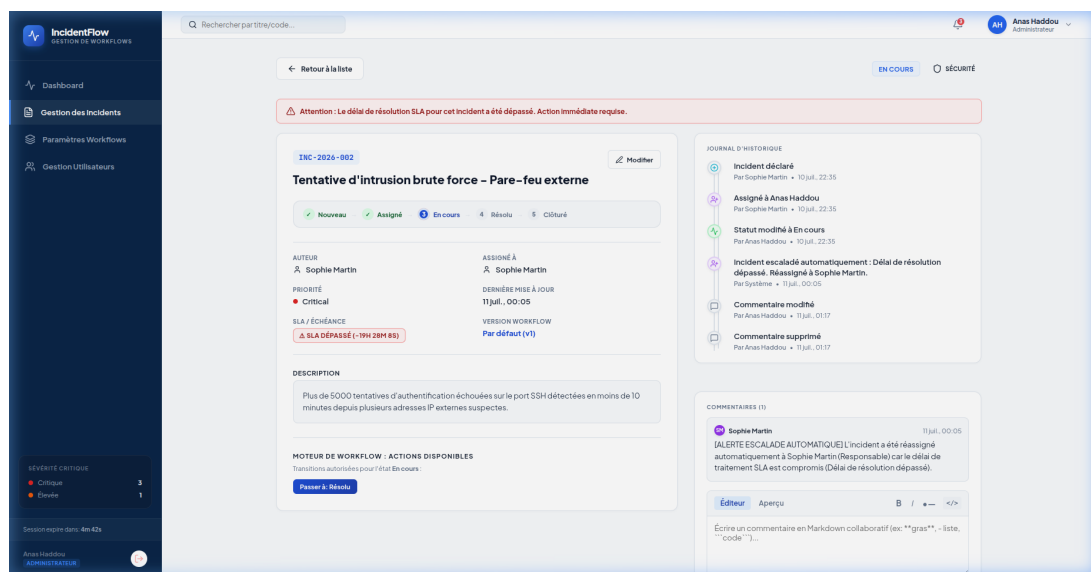


FIGURE 9 – Figure 7.2 — Vue détaillée de l'incident avec timeline collaborative et stepper

7.2.4 Liste de Recherche et de Filtrage des Incidents

Pour faciliter le travail des opérateurs de support de Netmar, un écran dédié liste l'ensemble des tickets.

- **Recherche et tris croisés** : Recherche plein texte avec boutons d'export CSV et PDF.
- **Compteurs de résultats** : Nombre dynamique d'éléments correspondants.

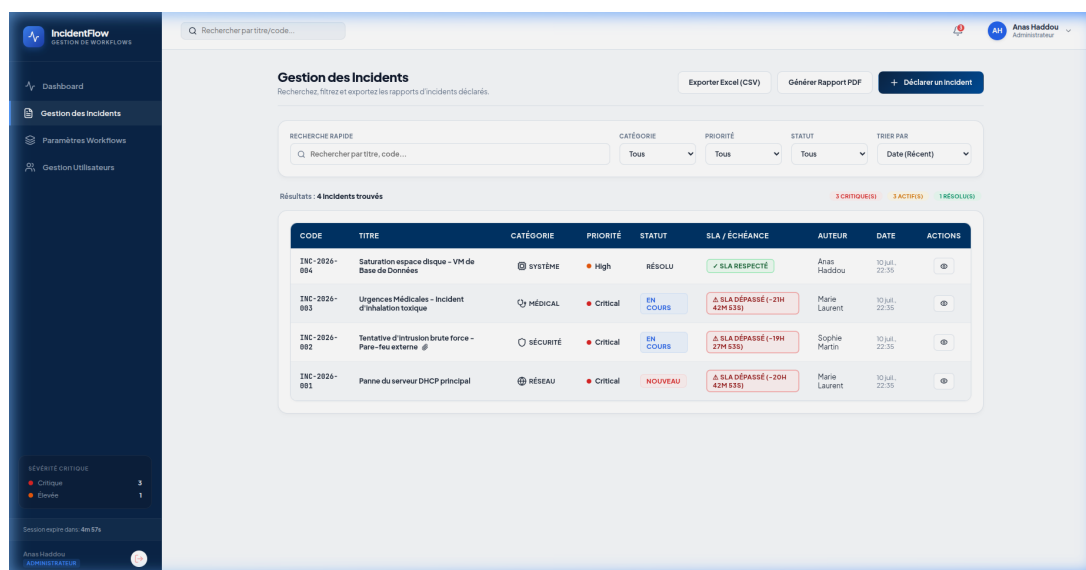


FIGURE 10 – Figure 7.3 — Écran de recherche multicritère et liste d'incidents

7.2.5 Palette de Commandes Universelle (Ctrl+K)

Interface de productivité de type modal activée globalement.

- **Raccourci clavier** : Intercepte la combinaison **Ctrl+K** ou **Cmd+K**.
- **Action** : Recherche instantanée floue sur le code incident ou commande système (ex : `> theme light`).

7.2.6 Éditeur Markdown de Commentaires

Composant d'édition collaboratif.

- **Responsabilité** : Permettre aux techniciens de formater leurs diagnostics (blocs de code d'erreurs, commandes système) de manière lisible.
- **Sécurité** : Analyseur regex échappant les scripts HTML pour empêcher les injections XSS.

7.2.7 Administration et Gestion des Utilisateurs

Afin de permettre aux administrateurs système de gérer les habilitations, un écran d'administration est mis à disposition. Il affiche la liste complète des comptes, les rôles affectés, les services et l'état d'activité du compte.

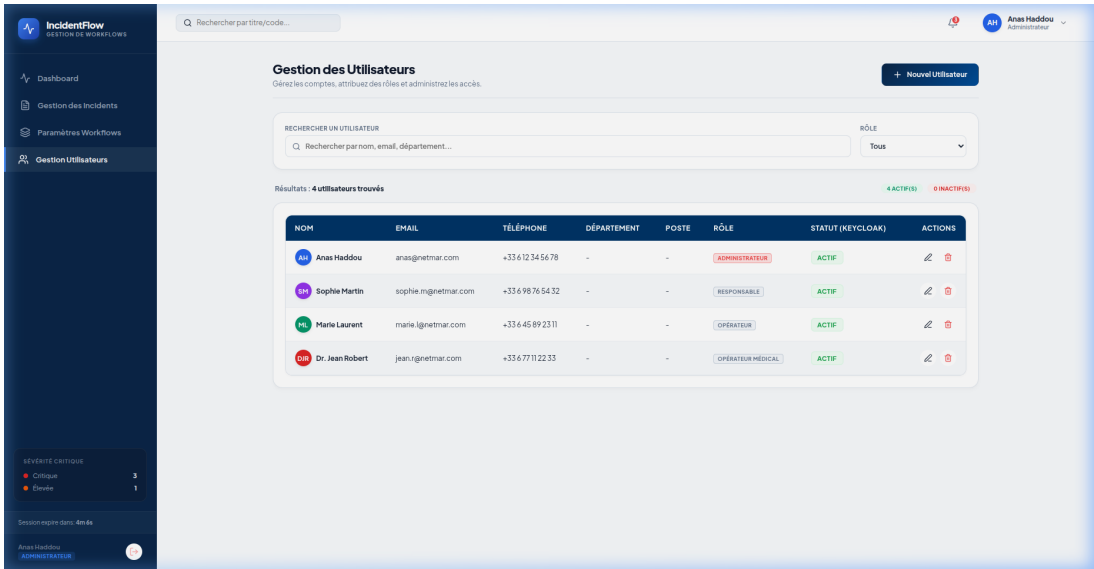


FIGURE 11 – Figure 7.4 — Écran d’administration des utilisateurs et rôles

8 Infrastructure DevOps et Orchestration

8.1 Configuration Multi-Conteneurs Docker Compose

La plateforme automatise le lancement de ses couches de persistance, de cache, d'identité et applicatives via le fichier `docker-compose.yml` :

```
1 # Extrait conceptuel de la structure du docker-compose.yml
2 services:
3   incidentflow-postgres:
4     image: postgres:16-alpine
5     container_name: incidentflow-postgres
6     ports:
7       - "5433:5432"
8     volumes:
9       - postgres_data:/var/lib/postgresql/data
10    environment:
11      POSTGRES_DB: IncidentFlow_db
12      POSTGRES_USER: IncidentFlow_user
13      POSTGRES_PASSWORD: ${DB_PASSWORD}
14
15    incidentflow-redis:
16      image: redis:7-alpine
17      container_name: incidentflow-redis
18      ports:
19        - "6380:6379"
20
21    incidentflow-backend:
22      build: ./backend
23      container_name: incidentflow-backend
24      ports:
25        - "8080:8080"
26      depends_on:
27        - incidentflow-postgres
28        - incidentflow-redis
29      environment:
30        SPRING_PROFILES_ACTIVE: prod
31        SPRING_DATASOURCE_URL: jdbc:postgresql://incidentflow-
           ↪ postgres:5432/IncidentFlow_db
```

8.2 Recette et Manifestes Kubernetes

Pour la transition vers un environnement Kubernetes (Production), le déploiement est structuré en plusieurs manifestes YAML :

- **Deployments** : Configurent le nombre de répliques des pods backend et frontend avec mise à l'échelle automatique.
- **Services** : Exposent les pods en interne via `ClusterIP` ou vers l'extérieur avec des contrôleurs `LoadBalancer` ou `NodePort`.
- **ConfigMap** et **Secrets** : Déplacent les configurations non sensibles et les mots de passe hors de l'image logicielle pour satisfaire les exigences de sécurité d'un cycle de release.
- **Ingress Controller** : Gère la couche de routage HTTP/HTTPS externe et la distribution TLS vers le bon service.

8.3 Résolution des Verrous Techniques de Déploiement

Durant la phase d'intégration, trois blocages majeurs ont été surmontés :

1. **Proxy réseau sur Docker Build** : L'environnement de construction réseau local bloquant le téléchargement des dépendances Maven, le **Dockerfile** du backend a été réécrit pour copier directement l'archive JAR compilée localement à l'aide de Maven sur la machine hôte.
2. **Problème de requêtes CORS** : L'authentification Vite ayant déplacé le port frontend de 3000 à 3001, la configuration CORS de **SecurityConfig.java** a été mise à jour pour accepter explicitement l'accès depuis ce port.
3. **Exception LazyInitializationException Hibernate** : Lors du rendu de la liste des workflows configurés, une erreur 500 survenait dans le backend. L'exception signalait l'absence de session Hibernate active pour charger les collections enfants de transitions et d'états d'un workflow. Ce verrou a été levé en configurant les relations JPA en chargement immédiat (**fetch = FetchType.EAGER**) sur ces collections.

9 Tests, Validation et Résultats

9.1 Tests Unitaires (JUnit 5 & Mockito)

La logique de validation des graphes de workflows et les calculs temporels de SLAs font l'objet d'une couverture de tests unitaires rigoureuse.

- **JUnit 5** valide que les structures attendues (présence de l'état initial et final) soient présentes.
- **Mockito** simule les réponses des repositories sans solliciter l'accès réseau ou le serveur SQL pour accélérer l'exécution des tests dans la chaîne d'intégration.

Voici l'exemple du cas de test `WorkflowValidationTest.java` validant la détection logique d'états isolés (orphelins) :

```
1 @Test
2 public void testOrphanStateThrowsException() {
3     Workflow workflow = new Workflow();
4
5     List<WorkflowState> states = new ArrayList<>();
6     states.add(WorkflowState.builder().name("Nouveau").build());
7     states.add(WorkflowState.builder().name("Assigne").build());
8     states.add(WorkflowState.builder().name("Orphelin").build()); //
9     ↪ Etat isole
10    states.add(WorkflowState.builder().name("Cloture").build());
11    workflow.setStates(states);
12
13    List<WorkflowTransition> transitions = new ArrayList<>();
14    transitions.add(WorkflowTransition.builder().fromState("Nouveau").
15    ↪ toState("Assigne").build());
16    transitions.add(WorkflowTransition.builder().fromState("Assigne").
17    ↪ toState("Cloture").build());
18    workflow.setTransitions(transitions);
19
20    Exception exception = assertThrows(IllegalArgumentException.class,
21    ↪ () ->
22    workflowService.validateWorkflowGraph(workflow)
23    );
24
25    assertTrue(exception.getMessage().contains("Orphelin"), "L'erreur
26    ↪ doit mentionner l'etat orphelin");
27 }
```

9.2 Tests d'Intégration avec Testcontainers

Pour s'assurer que les requêtes SQL complexes et les configurations de contraintes s'exécutent correctement, nous utilisons le framework **Testcontainers**. Lors du déclenchement des tests Maven, le système instancie un véritable conteneur PostgreSQL éphémère et vierge dans Docker. Le backend exécute ses requêtes d'intégration directement sur cette instance, garantissant des tests d'intégration réalistes et isolés.

10 Perspectives et Intégration d’IA (Classification Automatique)

10.1 Opportunités Fonctionnelles de l’IA

Une limitation du processus actuel réside dans la saisie du formulaire : l'utilisateur qualifie manuellement la catégorie et la priorité de sa demande, générant fréquemment des erreurs d'affectation et surchargeant le support. L'intégration de l'IA vise à automatiser ce tri à la source.

10.2 Comparatif Technique : Google Gemini vs Ollama

TABLE 1 – Comparatif des approches d'intégration IA

Critère	Option A : Google Gemini (Gratuit)	Option B : Ollama (Modèle Local)
Modèle cible	Gemini 1.5 Flash	Mistral 7B / Llama 3
Coût	Gratuit (Quota : 15 RPM / 1 500 RPD)	Gratuit (Open-source)
Calcul requis	Distant (Cloud Google)	Local (Serveur dédié $\geq$ 16 Go RAM)
Latence moyenne	1,5 à 2 secondes	5 à 15 secondes selon GPU
Faisabilité projet	Immédiate (API clé simple)	Complexe (Administration d'infrastructure)

10.3 Flux de Données et Conception de l’Endpoint

Le backend expose un point d'accès `POST /api/ai/suggest`. Lors d'une saisie utilisateur, le backend formule un prompt structuré incluant les listes de catégories et de priorités admissibles pour forcer l'IA à retourner un objet JSON exploitable en retour.

10.4 Mécanisme de Cache et Résilience

Pour prévenir le dépassement du quota de l'API gratuite Gemini (15 RPM) :

- Cache Redis** : Une signature (Hash) de la description saisie est calculée. Si un autre incident présente une description identique, Redis renvoie immédiatement la classification en cache sans appeler Gemini.
- Tolérance aux pannes** : En cas d'erreur de connexion à l'API Gemini, le backend intercepte l'exception et renvoie une suggestion par défaut, laissant le formulaire disponible pour saisie manuelle.

11 Conclusion et Bilan du Projet

Le projet **IncidentFlow** répond avec succès au besoin d'agilité de la gestion des processus métiers de l'entreprise Netmar. Grâce au développement d'un moteur de workflow dynamique validé par des algorithmes de parcours de graphe (DFS) et d'un système collaboratif étendu (observateurs, mentions, SLA synchrone, palette de commandes), la plateforme offre un outil robuste et moderne.

L'architecture conteneurisée gérée par Docker et planifiée sous Kubernetes garantit sa scalabilité opérationnelle, faisant d'IncidentFlow une solution stable et pérenne prête à l'exploitation.